THE COMPLETE PROMPT ENGINEERING RULEBOOK

Gemini 3.8 Flash — A Comprehensive Guide to Effective Prompting

Model: Gemini 3.8 Flash
Model ID: `gemini-3.8-flash`
Version: 1.0
Purpose: Achieving accurate, efficient, consistent and high-quality AI outputs
Scope: General-purpose Gemini 3.8 Flash, with a separate section for Gemini 3.8 Flash TTS.

PART 1 — UNDERSTANDING GEMINI 3.8 FLASH

1.1 What is Gemini 3.8 Flash?

Gemini 3.8 Flash is designed as a high-performance, general-purpose model for:

- Software engineering.
- Complex reasoning.
- Autonomous agents.
- Multimodal processing.
- Enterprise workflows.
- Research and analysis.
- Multi-step problem-solving.
- Tool-assisted tasks.

It combines relatively fast processing with the ability to handle more complex tasks through configurable thinking levels.

According to the supplied model documentation, its specifications include:

Specification	Gemini 3.8 Flash
Model ID	`gemini-3.8-flash`
Input types	Text, images, audio, video and PDFs
Output type	Text
Input context window	1,048,576 tokens
Maximum output	65,536 tokens
Thinking levels	LOW, MEDIUM, HIGH
Default thinking level	MEDIUM

Function calling	Supported
Structured outputs	Supported
Code execution	Supported
Computer use	Preview
File search	Supported
Google Search grounding	Supported
Google Maps grounding	Supported
URL context	Supported

These are model specifications, not guarantees that every capability is available in every interface or API configuration.

1.2 Understanding Gemini's Prompting Behaviour

Gemini 3.8 Flash benefits from a prompting approach based on:

1. Direct instructions.
2. Consistent prompt structure.
3. Explicit definitions.
4. Clear context placement.
5. Deliberate verbosity control.
6. Explicit multimodal references.
7. Appropriate thinking-level selection.
8. Well-defined tool-use policies.
9. Structured output requirements.
10. Verification and completion criteria.

The most important principle is:

Describe the task, provide the necessary information, establish boundaries and define the expected result.

Avoid unnecessarily complicated prompts that repeat instructions or prescribe obvious operations.

1.3 The Fundamental Prompting Philosophy

Gemini 3.8 Flash should be prompted with operational clarity rather than decorative instructions.

For example:

Weak:

"You are the world's greatest programmer. Think extremely deeply and create an amazing application."

Better:

"Develop a Python Flask application that allows users to upload CSV files, validate transactions, categorise expenses and generate monthly financial reports. Use SQLite for persistent storage and provide complete implementation instructions."

The second prompt defines an actual task and a measurable outcome.

PART 2 — THE CORE RULES OF PROMPT ENGINEERING

RULE 1: Be Precise and Direct

Clearly state what you want the model to accomplish.

Avoid vague instructions such as:

- Explain technology.
- Analyse the market.
- Improve this code.
- Research artificial intelligence.
- Make this document better.

Instead, specify the exact task.

Example:

"Analyse the adoption of electric vehicles in Karnataka between 2022 and 2026. Evaluate market growth, charging infrastructure, purchase incentives, operating costs and major adoption barriers. Support important findings with reliable sources."

Best practices

- Use direct action-oriented language.
- State the actual objective.
- Identify the subject and scope.
- Specify the intended result.
- Remove decorative or persuasive wording.
- Avoid unnecessary repetition.

The model should understand the task without having to interpret exaggerated descriptions.

RULE 2: Use One Consistent Prompt Structure

Gemini supports clearly structured prompts using Markdown headings or XML-style delimiters.

Both approaches can be useful.

Method A: Markdown

```
# TASK
Analyse the supplied dataset.

# CONTEXT
The dataset contains monthly sales records.

# REQUIREMENTS
- Identify missing values.
- Calculate monthly revenue.
- Identify sales trends.

# OUTPUT
Provide a structured analytical report.
```

Method B: XML-style structure

```
<task>
Analyse the supplied dataset.
</task>

<context>
The dataset contains monthly sales records.
</context>

<requirements>
Identify missing values.
Calculate monthly revenue.
Identify sales trends.
</requirements>

<output>
Provide a structured analytical report.
</output>
```

Important rules

1. Choose one primary structure.
2. Maintain consistent headings.
3. Separate instructions from source material.
4. Use clear section boundaries.
5. Avoid mixing multiple formatting systems without a reason.

For most ordinary prompts, Markdown headings are sufficient.

RULE 3: Define Ambiguous Terms and Parameters

Words such as the following can have different interpretations:

- High priority.
- Critical.
- Relevant.
- Recent.
- Significant.
- Concise.
- Safe.
- High confidence.
- Enterprise-ready.
- Similar.

When these terms affect the result, define them.

Example:

Weak:

"Classify the issues according to severity."

Better:

```
SEVERITY DEFINITIONS

HIGH:
The issue prevents users from accessing a major application feature.

MEDIUM:
The feature is available but experiences substantial limitations.

LOW:
The issue is cosmetic or has minimal impact on functionality.
```

When to use this rule

Use explicit definitions when:

- Building classification systems.
- Evaluating business opportunities.
- Comparing products.
- Analysing datasets.
- Designing automated workflows.
- Applying decision-making criteria.
- Developing AI agents.

Do not define ordinary words unnecessarily. Focus on terms that could materially change the result.

RULE 4: Place Critical Instructions First

Put essential behavioural rules and constraints near the beginning of the prompt.

For example:

```
# CRITICAL RULES

1. Use only the supplied source documents.
2. Do not invent statistics or references.
3. Identify contradictions between sources.
4. Clearly distinguish facts from assumptions.
5. Do not modify existing files unless explicitly authorised.

# CONTEXT

[Insert documents and supporting information]

# TASK

Analyse the supplied documents and identify their major findings.
```

This is particularly useful when working with large documents, codebases or datasets.

Critical instructions should not be hidden among large blocks of supporting material.

For persistent application-level behaviour, place suitable rules in the system instruction rather than repeating them in every request.

RULE 5: Control Verbosity Explicitly

Gemini 3 models generally favour direct and relatively efficient responses.

If you require detailed explanations, specify what the detail should cover.

Weak:

"Explain everything in detail."

Better:

"Provide a comprehensive explanation covering the definition, fundamental concepts, working principles, examples, practical applications, limitations and important distinctions."

You can also specify length:

- Maximum 100 words.
- Approximately 500 words.
- 1,000–1,500 words.
- Detailed technical report.
- Short executive summary.

Important distinction

Verbosity is not just word count.

It also concerns:

- Depth of explanation.
- Amount of supporting evidence.
- Technical detail.
- Number of examples.
- Background information.
- Level of interpretation.

Define the content that must be preserved when asking for a shorter answer.

RULE 6: Select the Appropriate Thinking Level

Gemini 3.8 Flash supports three thinking levels.

Thinking level	Suitable tasks	General trade-off
LOW	Fast responses, simple drafting, basic extraction, routine analysis	Lower reasoning overhead and faster responses
MEDIUM	Complex coding, general analysis, multi-step tasks and agents	Balanced performance
HIGH	Difficult mathematics, deep reasoning and complex multi-step problems	More reasoning effort and potentially greater latency and consumption

<IMPORTANT NOTE> The default thinking level is MEDIUM.

`MINIMAL` is not a supported thinking level for Gemini 3.8 Flash according to the supplied developer documentation.
</IMPORTANT NOTE>

LOW — When to use it

Use LOW for:

- Simple questions.
- Basic summarisation.
- Routine data extraction.
- Classification with clear rules.
- Simple content generation.
- Latency-sensitive applications.

MEDIUM — When to use it

Use MEDIUM for:

- Most programming tasks.
- Data analysis.
- Research synthesis.

- General business analysis.
- Multi-step problem-solving.
- Standard agentic workflows.

HIGH — When to use it

Use HIGH for:

- Complex mathematical reasoning.
- Difficult algorithmic problems.
- Complex software architecture.
- Multi-constraint optimisation.
- Deep technical investigations.
- Long and difficult multi-step workflows.

Important rules

1. Do not automatically use HIGH for every task.
2. Do not assume HIGH guarantees a correct answer.
3. Select thinking level based on task complexity.
4. Evaluate different levels on representative tasks when performance matters.
5. Do not try to change the API thinking configuration merely by writing "use HIGH thinking" in a prompt.

Thinking level is an API configuration, not simply a prompt-writing technique.

RULE 7: Put Large Context Before the Main Question

This is one of the most important Gemini-specific prompting recommendations.

For large-context tasks, provide the source material first and place the specific task or question at the end.

Recommended structure

```
# CRITICAL RULES

[Essential instructions]

# CONTEXT

[Document 1]

[Document 2]

[Dataset]

[Additional information]

# TASK
```

```
Based on all the information provided above, perform the following
analysis:

[Specific question or task]

# OUTPUT

[Expected format]
```

Why this structure matters

When working with long documents, the model needs to understand the supplied information before applying the final task.

A clear transition between context and task helps establish where the supporting material ends and the actual instruction begins.

Use a clear transition

Examples:

- "Based on the documents provided above..."
- "Using only the evidence supplied in the preceding sections..."
- "Now analyse the information presented above..."
- "Considering all the context provided, perform the following task..."

Important long-context rules

- Include relevant information rather than everything available.
- Remove duplicate documents.
- Identify the authoritative source.
- Clearly label different files.
- Identify contradictions.
- Place the final question after the large context.
- Use an explicit transition between context and task.

A large context window does not eliminate the need for context management.

RULE 8: Prompt Multimodal Inputs Explicitly

Gemini 3.8 Flash supports multiple input modalities.

These include:

- Text.
- Images.
- Audio.
- Video.
- PDF documents.

When supplying multiple files or media items, identify each input clearly.

Weak:

"Analyse these files."

Better:

```
# INPUT MATERIALS

Image A:
Screenshot of the current application interface.

Image B:
Screenshot of the proposed interface.

PDF:
Existing design guidelines.

Video:
Recording of the application running on a mobile device.

# TASK

Compare the proposed interface against the current interface.

Evaluate:
1. Visual hierarchy.
2. Navigation.
3. Accessibility.
4. Mobile usability.
5. Consistency with the supplied design guidelines.

# EVIDENCE REQUIREMENTS

Distinguish directly observable differences from recommendations.

Do not claim a design violation unless the relevant rule is present in the
supplied PDF.

# OUTPUT

Provide observations, issues, recommendations and priorities.
```

Image prompting

Specify:

- What the image represents.
- Which region to examine.
- What details to identify.
- What comparison to perform.
- What conclusions are required.

Video prompting

Specify:

- Relevant timestamps.
- Actions to observe.
- Events to identify.
- Sequence of events.
- Required analysis.

Audio prompting

Specify:

- Whether transcription is required.
- Whether speaker identification is relevant.
- Whether tone or delivery should be analysed.
- Whether a summary or detailed analysis is required.

PDF prompting

Specify:

- Relevant pages or sections.
- Required information.
- Whether tables and figures should be examined.
- Whether conclusions must be supported by specific passages.

Do not refer to multiple attachments simply as "this". Label them individually.

PART 3 — ADVANCED PROMPTING TECHNIQUES

RULE 9: Use Zero-Shot Prompting First

Zero-shot prompting means giving the model an instruction without providing examples.

Example:

```
Identify the programming languages mentioned in the following paragraph.

Text:
"""
The application was developed using Python, JavaScript and SQL.
"""

Return only the programming languages.
```

Use zero-shot prompting for straightforward tasks.

If the output is already correct and consistent, additional examples are unnecessary.

RULE 10: Use Few-Shot Prompting When Necessary

Few-shot prompting means providing a small number of examples to demonstrate the desired behaviour.

Example:

```
# TASK

Classify the following messages into one of three categories:
Billing, Access or Bug.

# EXAMPLES

Input:
"My card was charged twice."

Output:
Billing

Input:
"I cannot reset my password."

Output:
Access

Input:
"The application crashes whenever I open a project."

Output:
Bug

# NEW INPUT

"I paid successfully, but my account still shows the free plan."

# OUTPUT

Return one category only.
```

When few-shot prompting is useful

- Classification.
- Data extraction.
- Specific writing styles.
- Complex formatting.
- Decision boundaries.
- Repeated transformations.
- Consistent output requirements.

Important rules

1. Start with zero-shot.

2. Add examples only when they improve clarity.
3. Use representative examples.
4. Include difficult or borderline cases when necessary.
5. Keep examples consistent.
6. Avoid excessive examples.

More examples do not automatically produce better results.

RULE 11: Use Task Decomposition

Break complex objectives into smaller components when doing so improves reliability.

Example: Developing a financial application.

```
PHASE 1: Requirements analysis.

PHASE 2: System architecture.

PHASE 3: Database design.

PHASE 4: Backend development.

PHASE 5: Frontend development.

PHASE 6: Integration.

PHASE 7: Testing.

PHASE 8: Deployment.
```

This approach is useful for:

- Software development.
- Research.
- Business analysis.
- Large data projects.
- Complex mathematical problems.
- Agentic workflows.

However, avoid forcing unnecessary stages for simple tasks.

RULE 12: Use Comparative Prompting

When evaluating alternatives, define the criteria.

Example:

```
# TASK

Compare Python Flask and Django for a student-developed financial
application.

# EVALUATION CRITERIA
```

```
1. Learning difficulty.
2. Development speed.
3. Performance.
4. Database integration.
5. Security.
6. Scalability.
7. Deployment complexity.
8. Maintenance.

# OUTPUT

Present a comparison table.

Recommend the more suitable framework based on the stated requirements.

Explain the trade-offs.
```

Avoid asking which option is universally better. Ask which option is better for the specified circumstances.

RULE 13: Use Explicit Assumption Handling

Incomplete information does not always require a clarification question.

Tell the model when it should ask and when it should proceed.

Example:

```
# ASSUMPTION POLICY

If information is missing:

1. Determine whether the missing information materially affects the result.
2. If it affects an important decision, ask a targeted question.
3. If it does not materially affect the result, proceed using a reasonable
assumption.
4. Clearly state important assumptions.
5. Never invent critical facts, identifiers, financial values or
permissions.
```

This prevents unnecessary interruptions while maintaining reliability.

RULE 14: Use Verification Instructions

For important tasks, define what needs to be checked.

Example:

```
# VERIFICATION

Before finalising:

- Check whether all requirements are satisfied.
- Verify calculations.
```

```
- Identify logical inconsistencies.
- Check relevant edge cases.
- Confirm that the conclusion is supported by evidence.
- Identify unresolved limitations.

Do not claim that an operation was tested or completed unless it actually
was.
```

Verification is especially important for programming, research, calculations and external actions.

RULE 15: Use Clear Stop Conditions

Long-running workflows need a defined completion point.

Example:

```
# COMPLETION CRITERIA

The task is complete when:

1. All mandatory requirements are satisfied.
2. The requested deliverables are prepared.
3. Important results have been verified.
4. Remaining limitations are documented.

Stop when these conditions are met.

Do not continue performing unnecessary research or unrelated modifications.
```

This is particularly useful for autonomous agents.

PART 4 — RESEARCH AND INFORMATION ACCURACY

RULE 16: Define the Research Objective

Avoid broad research requests.

Instead of:

"Research AI in healthcare."

Use:

"Evaluate the adoption of artificial intelligence in diagnostic imaging in India, focusing on clinical applications, regulatory requirements, adoption barriers, costs and available evidence of effectiveness."

RULE 17: Define the Source Hierarchy

Specify which sources should be prioritised.

Recommended hierarchy:

1. Official government publications.
2. Official institutional documentation.
3. Peer-reviewed academic research.
4. University publications.
5. Official industry reports.
6. Reputable news publications.
7. Expert analysis.
8. Community discussions.

The source hierarchy should change according to the subject.

For technical documentation, official product documentation should generally receive priority.

For community experiences, relevant discussion forums can provide supplementary qualitative evidence.

RULE 18: Define the Research Period

For changing information, specify the relevant dates.

Examples:

- Current information as of November 2026.
- Developments from 2024–2026.
- Research published within the last three years.
- Historical trends from 2015–2025.

Require current verification where necessary.

RULE 19: Separate Facts From Inferences

Require the model to distinguish:

Verified facts: Directly supported by reliable sources.

Estimates: Values calculated using assumptions.

Inferences: Interpretations derived from evidence.

Opinions: Subjective evaluations.

Recommendations: Suggested actions based on the available information.

This is essential for reliable research.

RULE 20: Require Evidence and Citations

Example:

```
Support major factual claims with reliable sources.

For important findings:

- Provide the supporting evidence.
- Identify the source.
- Include the relevant publication date.
- Provide a citation.
- Identify uncertainties.

Do not fabricate citations, statistics or quotations.

If a claim cannot be verified, clearly disclose that limitation.
```

RULE 21: Request Limitations and Counterarguments

A research report should not simply support one conclusion.

Ask for:

- Supporting evidence.
- Contradictory evidence.
- Alternative explanations.
- Data limitations.
- Research limitations.
- Potential biases.
- Unresolved questions.

This makes the analysis more balanced.

PART 5 — PROGRAMMING AND SOFTWARE DEVELOPMENT

RULE 22: Specify the Development Environment

For coding tasks, provide:

- Operating system.
- Programming language.

- Language version.
- IDE.
- Frameworks.
- Libraries.
- Database.
- Deployment environment.

Example:

```
ENVIRONMENT

Operating System: Windows 11
IDE: Visual Studio Code
Language: Python
Framework: Flask
Database: SQLite
Frontend: HTML, CSS, JavaScript
```

RULE 23: Define the Existing Project Structure

When modifying an existing application, explain which files are involved.

Example:

```
# PROJECT STRUCTURE

project/
    app.py
    database.db
    templates/
    static/
    requirements.txt

# MODIFICATION RULES

Modify only app.py.

Preserve:
- Existing database records.
- Existing routes.
- Existing participant information.
- Existing application functionality.

Do not modify unrelated files.
```

This is particularly important when the model works with uploaded project files.

RULE 24: Specify the Required Code Quality

Define whether you need:

- Beginner-friendly code.
- Minimal code.
- Readable code.

- Modular code.
- Optimised code.
- Production-oriented code.

Do not request advanced architecture when a simple implementation is sufficient.

RULE 25: Define Error Handling

Specify how the application should handle:

- Invalid inputs.
- Missing files.
- Duplicate records.
- Database failures.
- Unsupported file formats.
- Empty datasets.
- Network failures.
- Unexpected user actions.

Example:

"Validate uploaded CSV files before processing. Handle missing columns, invalid values and duplicate records with meaningful error messages."

RULE 26: Request Testing and Verification

For important programming tasks, request:

1. Syntax verification.
2. Logic verification.
3. Input validation.
4. Edge-case testing.
5. Database verification.
6. Integration testing.
7. Relevant regression tests.

Where execution tools are available, prefer actual testing over code inspection alone.

Do not accept claims of successful testing without evidence.

PART 6 — DATA ANALYSIS AND MATHEMATICAL PROBLEM-SOLVING

RULE 27: Define the Dataset and Objective

For data analysis, provide:

- Dataset description.
- File format.
- Column information.
- Number of records, if known.
- Analysis objective.
- Expected outputs.

RULE 28: Specify the Analysis Requirements

Example:

```
# TASK

Analyse the supplied CSV dataset using Python and Pandas.

# REQUIREMENTS

1. Inspect the dataset.
2. Identify missing values.
3. Identify duplicates.
4. Validate data types.
5. Clean the data.
6. Perform relevant statistical analysis.
7. Identify important patterns.
8. Generate meaningful visualisations.
9. Present conclusions.

# IMPORTANT

Use appropriate missing-value treatment based on the data.

Do not apply mean imputation to categorical columns.

Do not invent numerical results.
```

RULE 29: Request Numerical Evidence

Avoid vague descriptions.

Instead of:

"Sales increased significantly."

Request:

"Calculate the percentage increase in sales, show the actual values used and identify the relevant period."

RULE 30: Structure Mathematical Solutions

For mathematical problems, specify:

1. Given information.
2. Required result.
3. Relevant formula.
4. Substitution.
5. Calculations.
6. Final answer.
7. Units.
8. Verification.

For complex problems, request derivations and alternative methods where useful.

PART 7 — BUSINESS AND DECISION-MAKING

RULE 31: Define the Decision

A business analysis should support an actual decision.

Example:

"Evaluate whether a ready-to-cook food business targeting college students in Mangaluru is financially feasible with an initial investment of ₹5,000."

RULE 32: Specify the Constraints

Include:

- Initial capital.
- Available time.
- Target customers.
- Geographical market.
- Operating costs.
- Expected revenue.
- Available resources.
- Risk tolerance.

RULE 33: Request Quantitative Analysis

Where relevant, calculate:

- Initial investment.
- Fixed costs.
- Variable costs.
- Revenue.
- Gross profit.
- Net profit.
- Break-even point.
- Return on investment.
- Payback period.

Clearly distinguish estimates from verified figures.

RULE 34: Use Scenario Analysis

Request three scenarios:

Optimistic: Favourable conditions.

Base: Reasonably expected conditions.

Pessimistic: Unfavourable conditions.

Ask the model to explain the assumptions behind each scenario.

RULE 35: Require a Justified Recommendation

A recommendation should consider feasibility, costs, risks and expected benefits.

Do not recommend an option simply because it has the highest projected revenue.

PART 8 — TOOL USAGE AND AGENTIC WORKFLOWS

Gemini 3.8 Flash supports tool-assisted and agentic workflows. These tasks require more than ordinary task instructions.

RULE 36: Define Tool-Use Conditions

Specify:

- Which tools may be used.
- When external research is necessary.
- Which operations are read-only.
- Which operations change external state.
- What information must be verified.

Example:

"Use official documentation to verify current API capabilities. Use code execution for calculations when numerical accuracy is important."

RULE 37: Establish Risk Boundaries

Different actions carry different levels of risk.

Lower-risk activities:

- Reading files.
- Searching documentation.
- Analysing information.
- Performing calculations.
- Drafting proposed changes.

Higher-risk activities:

- Sending emails.
- Making purchases.
- Deleting files.
- Changing account settings.
- Publishing content.
- Submitting forms.
- Modifying external databases.

Require explicit authorisation for consequential actions according to the application's permission rules.

RULE 38: Define Recovery Behaviour

Example:

```
# FAILURE HANDLING

If a tool fails:

1. Identify the type of failure.
2. Determine whether retrying is safe.
3. Check whether a previous operation succeeded.
4. Retry only when appropriate.
5. Record unresolved failures.
6. Never claim successful completion when the result is unknown.
```

RULE 39: Define Adaptability

Do not force the model to follow an initial plan when new evidence contradicts it.

Example:

"Adapt the investigation when new evidence disproves an earlier assumption. Preserve the original objective while updating the approach."

RULE 40: Define Stop Conditions

Tell the model when to stop.

Example:

"Stop when all requested actions are complete, all mandatory checks have been performed and any remaining limitations have been documented. Ask for additional authorisation if the next action requires it."

PART 9 — STRUCTURED OUTPUTS

RULE 41: Separate Semantic Rules From Output Schemas

Gemini 3.8 Flash supports Structured Outputs.

For applications that require machine-readable responses, use a response schema to enforce the output structure.

The prompt should define the meaning of the information.

The schema should define the format.

Example:

Prompt:

```
# TASK

Classify the supplied customer feedback.

# DECISION RULES

- Select one primary category.
- Severity represents actual product impact.
- Do not infer customer churn without evidence.
- Use unknown when information is insufficient.
```

Example JSON schema:

```
{
  "type": "object",
  "properties": {
    "category": {
```

```
      "type": "string",
      "enum": [
        "billing",
        "access",
        "bug",
        "feature_request",
        "other",
        "unknown"
      ]
    },
    "severity": {
      "type": "string",
      "enum": [
        "low",
        "medium",
        "high",
        "critical",
        "unknown"
      ]
    },
    "action_required": {
      "type": "boolean"
    },
    "reason": {
      "type": "string"
    }
  },
  "required": [
    "category",
    "severity",
    "action_required",
    "reason"
  ],
  "additionalProperties": false
}
```

The schema is an API configuration, not a substitute for a clear prompt.

Do not duplicate complicated schemas in natural-language instructions unless necessary.

PART 10 — COMMON PROMPTING MISTAKES

Mistake	Better approach
Vague objectives	Define the exact outcome
Excessive persuasive language	Use direct instructions
Mixing multiple prompt structures	Use one consistent structure
Undefined terminology	Specify decision boundaries
Critical rules buried in context	Place them first
Main question placed before large context	Place the final query after context
Including irrelevant documents	Provide only relevant material

Using HIGH for everything	Calibrate thinking level
Treating thinking level as a substitute for clarity	Improve the prompt itself
Vague multimodal references	Label each input
Giving tools without restrictions	Define tool policies
Relying on prompt prose for JSON validation	Use Structured Outputs
Excessive few-shot examples	Use representative examples
Requesting long internal reasoning traces	Request conclusions, evidence and necessary explanations
Missing failure-handling instructions	Define retry and recovery rules
No completion criteria	Define stop conditions
Changing prompts without testing	Evaluate prompt versions

PART 11 — THE PRACTICAL GEMINI 3.8 FLASH PROMPT FRAMEWORK

The following is a comprehensive framework for important tasks.

Not every prompt needs every component.

THE EIGHT-PART FRAMEWORK

1. CRITICAL RULES

Define important behavioural requirements, source priorities and restrictions.

2. CONTEXT

Supply relevant documents, code, data, images, videos and background information.

3. DEFINITIONS

Explain ambiguous terms, thresholds, categories and evaluation criteria.

4. TASK

State the precise objective or question.

5. TOOL AND ACTION POLICY

Specify tools, permissions, risk boundaries and recovery behaviour when relevant.

6. OUTPUT

Define format, length, structure, tone and mandatory information.

7. VERIFICATION

Explain what must be checked before completion.

8. STOP CONDITIONS

Define when the task is finished and when clarification or authorisation is required.

REUSABLE MASTER TEMPLATE

```
# CRITICAL RULES

[List essential instructions and restrictions.]

# CONTEXT

[Provide relevant background, files, datasets or source material.]

# DEFINITIONS

[Define important terms, categories and thresholds.]

# TASK

[Clearly state the result you want.]

# REQUIREMENTS

1. [Requirement 1.]
2. [Requirement 2.]
3. [Requirement 3.]

# TOOL AND ACTION POLICY

[Specify tools, permissions and boundaries if applicable.]

# OUTPUT

Format: [Report/table/code/JSON/etc.]

Structure:
[Required sections.]

Detail:
[Desired level of explanation.]

Audience:
[Intended reader.]

# VERIFICATION

[Specify what must be checked.]

# STOP CONDITIONS
```

```
[Define completion and clarification conditions.]
```

For ordinary tasks, use a shorter version.

```
# TASK

[What needs to be accomplished.]

# CONTEXT

[Relevant information.]

# REQUIREMENTS

[Important instructions.]

# OUTPUT

[Expected result and format.]
```

Use the smallest structure that adequately expresses the task.

PART 12 — API CONFIGURATION: IMPORTANT TECHNICAL DISTINCTIONS

This section is intended for developers integrating Gemini 3.8 Flash through an API.

Prompt instructions and API configuration are different things.

12.1 Thinking Level

The supplied developer documentation identifies the following valid values:

```
LOW
MEDIUM
HIGH
```

MEDIUM is the default.

MINIMAL is unsupported.

Configure the thinking level through the supported API configuration rather than relying on natural-language instructions.

12.2 Sampling Parameters

According to the supplied Gemini 3.8 Flash developer documentation, the following legacy sampling parameters are ignored by the backend:

- `temperature`
- `top_k`
- `top_p`

Do not rely on these parameters to control model behaviour for this model.

12.3 Unsupported Parameters

The supplied documentation identifies these parameters as unsupported and states that passing them causes API errors:

- `frequency_penalty`
- `presence_penalty`
- `candidate_count`

Remove them from incompatible legacy configurations.

12.4 Function Calling

Function responses must correctly match the preceding function call.

Ensure that the function-call identifier, function name and execution sequence are consistent.

12.5 Multimodal Inputs

Place multimodal assets in the appropriate request payload.

Do not assume that describing an image or video in plain text is equivalent to supplying the actual media.

12.6 Conversation History

The supplied developer documentation identifies these restrictions:

- History payloads cannot end with a model-role turn.
- Prefilled model turns are unsupported.
- Empty conversation turns may be dropped or cause validation errors.

Follow the actual API's supported conversation structure.

12.7 General API Recommendation

Use the current official SDK and documentation for the exact request syntax.

Do not copy old API parameters into a new model integration without checking compatibility.

PART 13 — PROMPT EVALUATION AND OPTIMISATION

A prompt should be evaluated based on actual performance rather than appearance.

13.1 Establish a Baseline

Start with a simple prompt.

Test it against representative tasks.

Record the results.

13.2 Measure Performance

Evaluate:

1. Task completion.
2. Instruction following.
3. Factual accuracy.
4. Evidence grounding.
5. Output consistency.
6. Formatting reliability.
7. Tool-use accuracy.
8. Error recovery.
9. Response latency.
10. Token consumption.
11. Overall cost.

13.3 Test Different Thinking Levels

When the task justifies it, compare:

- Same prompt + LOW.
- Same prompt + MEDIUM.
- Same prompt + HIGH.

Keep other variables unchanged during the comparison.

This helps determine whether additional reasoning effort actually improves the result.

13.4 Test Edge Cases

Do not test only straightforward examples.

Include:

- Incomplete information.
- Conflicting evidence.
- Invalid inputs.
- Unusual cases.
- Ambiguous requests.
- Large datasets.
- Tool failures.

13.5 Maintain Prompt Versions

When improving an important prompt:

1. Save the original.
2. Make one meaningful change.
3. Test the revised version.
4. Compare results.
5. Retain the better version.
6. Repeat the evaluation when the model or configuration changes.

A shorter prompt is not automatically better. A longer prompt is not automatically better either.

The objective is reliable performance.

PART 14 — GEMINI 3.8 FLASH TTS: SEPARATE PROMPTING GUIDELINES

This section applies specifically to the speech-generation models:

- `gemini-3.8-flash-tts`
- `gemini-3.8-flash-lite-tts`

These are separate from the general-purpose `gemini-3.8-flash` model.

The TTS models use a different prompting structure.

14.1 Understand the Three Components

Gemini 3.8 TTS separates voice identity, delivery and spoken content.

Component	Purpose
Voice	Who is speaking
`speech_metadata.style`	How the speaker delivers the current turn
`text`	What the speaker actually says

This separation is essential.

Do not place long acting instructions inside the spoken transcript.

14.2 Treat the Text Field as a Verbatim Transcript

The `text` field is intended to contain the exact words to be spoken.

For example:

```
Welcome everyone to today's session.

We are excited to have you here.
```

Do not insert ordinary stage directions such as:

```
Say this in an excited voice:
Welcome everyone!
```

The instruction may be interpreted as spoken content.

Instead, use the appropriate style field.

14.3 Use the Style Field for Sustained Delivery

Use `speech_metadata.style` to define delivery across an entire turn.

Examples:

```
casual and friendly
speaking slowly and clearly
whispering urgently
sarcastic and slightly amused
formal news presenter with controlled delivery
```

Style can control:

- Emotion.
- Speaking pace.
- Pitch.
- Prosody.
- Inflection.
- Overall delivery.

Keep style instructions short and relevant.

14.4 Use Inline Tags for Point-in-Time Vocal Events

Use angle brackets to place momentary vocal events at the appropriate position.

Examples:

```
<breath>
<sigh>
<laugh>
<gasp>
<short pause>
<long pause>
```

These tags are intended for human vocalisations and pauses, not arbitrary environmental sound effects.

14.5 Control Pacing and Pauses

You can control pauses in three ways.

Punctuation:

```
I thought about it... and then I understood.
```

Inline pause tags:

```
Wait a moment. <short pause> I have an idea.
```

Turn-level style:

```
speaking slowly
```

Use punctuation naturally before adding excessive pause tags.

14.6 Control Emotion and Pitch

Use the style field for sustained vocal characteristics.

Examples:

```
high pitch, cheerful and excited inflection
monotone and flat
calm, reassuring and warm
```

If the emotion changes significantly within a scene, divide the dialogue into separate turns or parts with their own style settings.

14.7 Emphasise Important Words

Use capitalisation and punctuation to emphasise words.

Example:

```
This is a VERY important announcement!
```

Do not capitalise entire paragraphs unnecessarily.

14.8 Use Natural Conversational Language

Write transcripts as people actually speak.

Natural disfluencies can improve conversational realism.

Example:

```
Oh, uh... yeah, I think that's interesting. Hmm, let me explain.
```

Avoid writing every sentence as formal written prose when the intended result is natural conversation.

14.9 Design Voice Personas Separately

Use Voice Design to create a persistent voice identity.

Describe relatively stable characteristics such as:

- Vocal timbre.
- Pitch.
- Regional accent.
- Baseline cadence.
- Diction.
- Vocal resonance.

Example:

"A mature male speaker with a warm, resonant baritone, clear diction, a measured speaking pace and a neutral Indian English accent."

Use the resulting voice ID across generations.

Do not repeatedly include long voice descriptions in the style field.

Do not use style instructions to change immutable speaker traits such as age, gender or permanent accent.

14.10 Maintain Voice Consistency

The supplied TTS documentation recommends relying on the configured voice reference for identity consistency.

Avoid unnecessary instructions such as:

- Never change your voice.
- Maintain identical timbre.
- Never switch speaker identity.

These instructions are generally unnecessary when the correct voice reference is configured.

14.11 Use Backchannels for Two-Speaker Conversations

For multi-speaker dialogue, pipe characters can represent short listener reactions.

Example:

```
So the launch is Thursday |oh really| Are we actually ready?
```

The reaction is spoken by the other speaker while the primary speaker continues.

Use short natural reactions such as:

- `|mhm|`
- `|oh really?|`
- `|wait|`
- `|yeah|`

The supplied documentation identifies this capability particularly for `gemini-3.8-flash-tts`.

14.12 Use Appropriate TTS Models

Model	Recommended use
Gemini 3.8 Flash TTS	Studio-grade acting, nuanced emotion, regional dialects, multi-speaker scenes and long-form narrative stability
Gemini 3.8 Flash-Lite TTS	High-volume speech generation, real-time agents, customer support and read-aloud applications

Both models use the same basic API structure and prompting format according to the supplied TTS documentation.

14.13 Recommended TTS Workflow

1. Select or design the voice.
2. Prepare a natural spoken transcript.
3. Test the transcript with an empty style field.
4. Listen to the baseline output.
5. Add a short style instruction only if necessary.
6. Add inline vocal tags at appropriate positions.
7. Divide major emotional changes into separate turns.

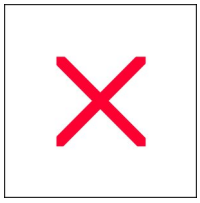
8. Test the final output.

The key principle is to separate the script from the performance direction.

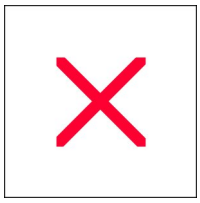
PART 15 — FINAL GEMINI 3.8 FLASH PROMPTING CHECKLIST

Before submitting an important prompt, verify the following.

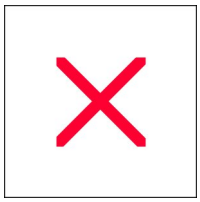
Objective and Context

- 

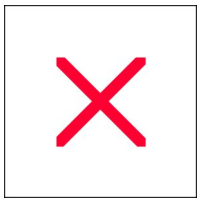
  Is the objective clearly defined?

- 

  Have I supplied relevant background?

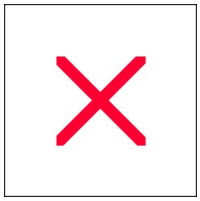
- 

  Is the scope specific?

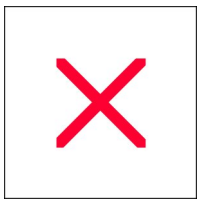
- 

  Have I removed irrelevant context?

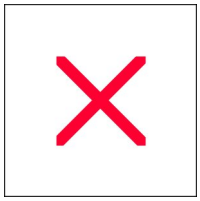
Prompt Structure

- 

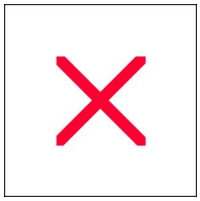
  Are the instructions organised?

- 

  Am I using one consistent formatting structure?

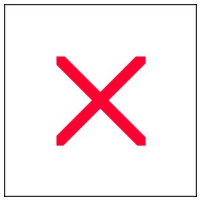
- 

  Are critical rules placed first?

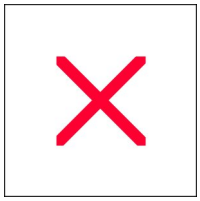
- 

  Are definitions included where necessary?

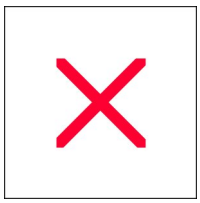
Thinking and Complexity

- 

  Have I selected the appropriate thinking level?

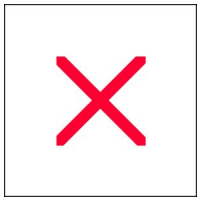
- 

Is LOW, MEDIUM or HIGH justified by the task?

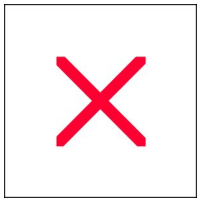
- 

Am I avoiding unnecessary complexity?

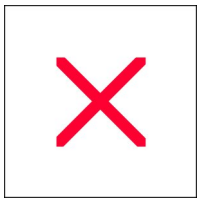
Long Context

- 

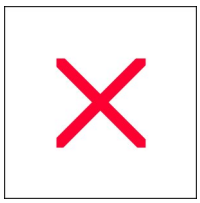
Is the supporting material clearly labelled?

- 

Have I placed large context before the final query?

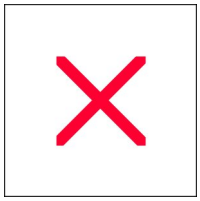
- 

Is there a clear transition between context and task?

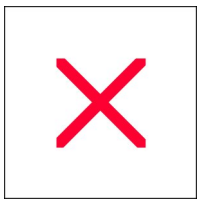
- 

Have I identified source priorities?

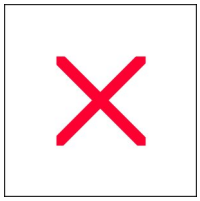
Multimodal Inputs

- 

  Have I identified each image, video, audio file or PDF?

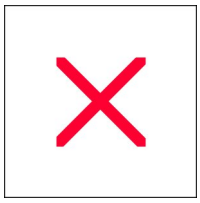
- 

  Have I explained how each input should be used?

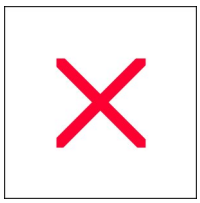
- 

  Have I distinguished observations from inferences?

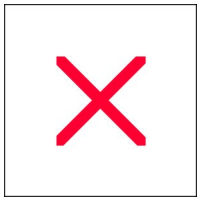
Output

- 

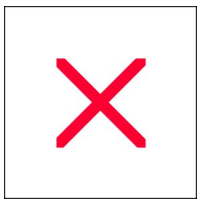
  Is the required format defined?

- 

  Is the expected level of detail clear?

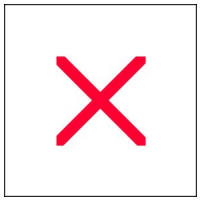
- 

Are mandatory sections specified?

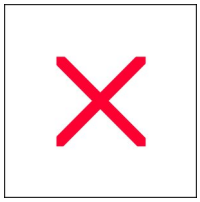
- 

Is the intended audience identified?

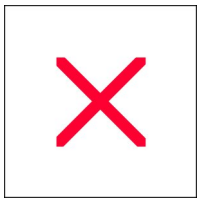
Reliability

- 

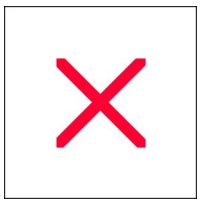
Are evidence requirements clear?

- 

Are assumptions handled appropriately?

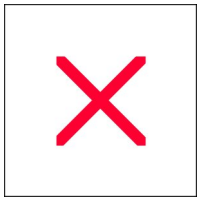
- 

Are verification requirements defined?

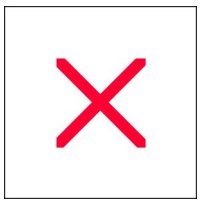
- 

Are limitations to be disclosed?

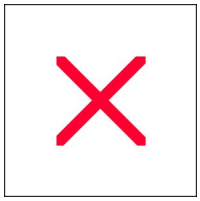
Tools and Agents

- 

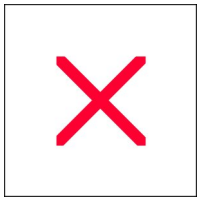
  Are tool-use conditions defined?

- 

  Are permissions and risk boundaries clear?

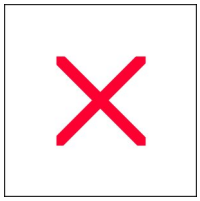
- 

  Is failure recovery specified?

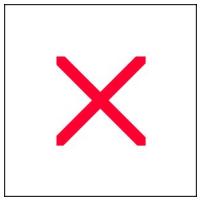
- 

  Are completion and stop conditions defined?

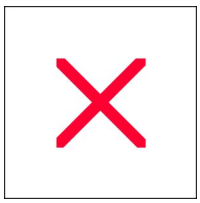
Evaluation

- 

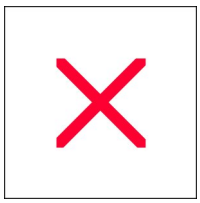
  Has the prompt been tested?

- 

Have representative edge cases been considered?

- 

Have different thinking levels been evaluated where useful?

- 

Are prompt versions maintained?

PART 16 — THE 25 GOLDEN RULES

1. Be direct and precise.
2. Define the exact outcome.
3. Supply relevant context.
4. Use one consistent prompt structure.
5. Place critical instructions first.
6. Define ambiguous terms and parameters.
7. Specify the desired output format.
8. Control verbosity explicitly.
9. Select the appropriate thinking level.
10. Do not use unsupported thinking levels.
11. Place large context before the final query.
12. Use a clear transition between context and task.
13. Avoid irrelevant or duplicated context.
14. Label multimodal inputs explicitly.
15. Start with zero-shot prompting.
16. Use few-shot examples when they clarify behaviour.
17. Break complex tasks into logical components when useful.
18. Define source priorities for research.
19. Distinguish verified facts from assumptions and inferences.
20. Specify the technical environment for programming.
21. Define tool permissions and risk boundaries.
22. Specify recovery behaviour for agentic tasks.
23. Separate semantic rules from structured output schemas.
24. Define verification and completion criteria.
25. Evaluate prompts using real representative tasks.

PART 17 — FINAL PROMPT ENGINEERING PHILOSOPHY

Gemini 3.8 Flash performs best when the task is communicated clearly, the relevant context is organised properly and the expected result is explicitly defined.

The objective is not to create the longest or most sophisticated-looking prompt.

It is to create a prompt that provides everything necessary without unnecessary instructions.

Remember the following:

A weak prompt identifies a topic.

A good prompt defines a task.

An excellent prompt defines the objective, supplies relevant context, establishes decision rules, sets boundaries and specifies what a successful result must contain.

For Gemini 3.8 Flash, remember these three particularly important principles:

1. **Prompt design:** Clearly define the task, context, requirements and output.
2. **Configuration:** Select the appropriate thinking level and available tools.
3. **Evaluation:** Test the result, identify weaknesses and refine the prompt.

Do not use HIGH thinking for every task. Do not fill the entire context window unnecessarily. Do not add examples without a reason. Do not prescribe every obvious step.

Use the smallest prompt and configuration that reliably accomplish the task.

That is the foundation of efficient Gemini 3.8 Flash prompt engineering.